Building an AP Automation Product from Scratch

The complete engineering document: miniERP + finErpAP

Every layer, every file, every technology choice - and why.

Sourav - July 2026

Contents

1. The Problem and the Product Vision

Every company that buys things receives vendor invoices. Processing them manually means: an employee opens the email, downloads the PDF, retypes its contents into the accounting system (ERP), checks it against the purchase order, chases a manager for approval, and finally records the payment. A mid-size company does this hundreds of times a month. It is slow, expensive, and error-prone.

The product we built (modeled on Procindex, YC S25) automates that entire loop:

```
vendor emails invoice PDF
  -> poller catches it (60s)
  -> AI reads any layout into clean JSON
  -> deterministic rules verify it against the PO in the ERP
  -> routing rules decide who must approve (small amounts: nobody)
  -> a human approves only the big or broken ones
  -> payment is recorded, the PO closes, everything is audited
```

The one principle behind the whole design

AI reads, rules decide. AI is used in exactly one step - turning a messy PDF into structured data - because that input is unpredictable and no regex can read every vendor's layout. The moment the data is clean, deterministic code takes over, because money must be exact, repeatable, and auditable. AI never decides what to pay. A second principle follows everywhere: **the ERP is the memory** - the single source of financial truth that every other component reads from or writes to.

2. Phase 1: The Mini-ERP - the System of Record

Why we built an ERP first

The real product integrates with an ERP the customer already owns (QuickBooks, SAP, Tally). We built our own minimal ERP first for three reasons: (1) to deeply understand the finance data model - vendors, POs, invoices, payments - instead of treating the ERP as a black box; (2) to have a free, local integration target with a real REST API; (3) to force the correct architecture - the product treats the ERP as a foreign system reachable only over HTTP, exactly like production.

Location: `Desktop/miniERP`, port 4000, database `mini_erp`. It is a complete standalone app: Express backend plus a single-file frontend (public/index.html) that the backend serves itself.

The stack, and why each piece

Technology	Why it was chosen
TypeScript	Type safety catches whole classes of bugs at compile time; the de-facto backend language for this product category.
Express	Minimal, unopinionated HTTP framework - nothing hidden, ideal for learning.
PostgreSQL	Finance data is relational (invoice -> vendor -> PO). ACID transactions are non-negotiable for money.
Prisma ORM	Schema-as-code (schema.prisma), generated type-safe client, migration history.
Zod	Validates every request body at the door; bad input never reaches the database.

The 4-file module pattern

Every business concept (vendors, customers, purchaseOrders, invoices, payments) is a folder with the same four files. Learn one module and you know them all:

```
routes.ts      which URL -> which controller          (the reception desk)
schema.ts      Zod validation of the input            (the form checker)
controller.ts  unpack request -> call service -> respond (thin glue)
service.ts     ALL business logic + database access   (the brain)
```

Three ideas baked into the ERP core

- **Multi-tenancy from day one.** Every table carries organizationId; every query in every service is scoped by it. Company A can never see Company B's rows. The tenant arrives via the x-org-id header (middleware/tenant.ts).
- **The invoice status state machine.** RECEIVED -> PENDING_APPROVAL -> APPROVED -> PAID (REJECTED is terminal). An ALLOWED_TRANSITIONS map in invoice.service.ts refuses illegal jumps such as RECEIVED -> PAID. The workflow is enforced by the system of record itself.
- **Atomic money writes.** Recording a payment inserts the Payment row, flips the invoice to PAID, and closes the PO inside one prisma.$transaction - all of it happens or none of it does.

Purchase orders and matching

A PO is the buyer's document ('I want 5 laptops for Rs. 2,50,000') created BEFORE any invoice. Its whole purpose is to be the thing the vendor's invoice is checked against later. PO lifecycle: OPEN -> MATCHED (an invoice with the same amount linked to it) -> CLOSED (that invoice was paid). The ERP

validates that a linked PO belongs to the same vendor as the invoice, and auto-marks the PO MATCHED when amounts are equal.

3. The ERP Database Design

Five tables. Money uses DECIMAL(14,2) - never floating point, which cannot represent 0.1 exactly and corrupts totals. Every table has organizationId plus created/updated timestamps.

Table	Purpose	Key design details
Vendor	Companies you buy from	unique(organizationId, name)
Customer	Companies you sell to	unique(organizationId, name)
PurchaseOrder	What you ordered	status OPEN/MATCHED/CLOSED/CANCELLED; unique(orgId, poNumber)
Invoice	Bills received (AP) or sent (AR)	type AP|AR; status machine; optional purchaseOrderId link; unique(orgId, type, invoiceNo) = duplicate-invoice guard enforced by the DB itself
Payment	Money that moved	links to its invoice; written atomically with status flips

The unique constraint on (organizationId, type, invoiceNo) is worth highlighting: even if every layer of application code fails, PostgreSQL itself makes it physically impossible to store the same vendor invoice twice. Defence in depth starts at the schema.

4. Phase 2: finErpAP - the Product Architecture

finErpAP is the sellable product: the automation layer around the ERP. It lives at `Desktop/finErpAP` as two independently deployable apps plus infrastructure:

```
finErpAP/
  server/   backend pipeline + API  (Express, port 5000, DB finerp_ap)
  web/      dashboard              (Next.js, port 3000)
  docker-compose.yml   Valkey queue broker (Docker, port 6379)
```

Why the product is separate from the ERP

They share no code and no database. The only bridge is HTTP through one adapter file, `server/src/erp/erpClient.ts` - the single file that knows how to talk to the ERP. Reason: in reality the customer's ERP already exists and belongs to them; a product that assumed shared code or a shared database could not exist. Supporting QuickBooks later means writing one new client file with the same function names - the pipeline never changes.

Why the dashboard is not Next.js API routes

The pipeline needs long-running components: a queue worker and a 60-second email poller. Next.js API routes are request-scoped - they run when a request arrives and stop. Background processing belongs in a persistent Node service (server/), while Next.js does what it is best at: the UI (web/).

The adapter pattern, proven three times

Every external dependency sits behind a small interface file, so swapping the implementation touches one file only. This paid rent three separate times in this project:

- Storage: local disk today, S3/Cloudflare R2 later - only lib/storage.ts changes.
- PDF parsing: the pdf-parse library failed on our files; swapping to pdfjs-dist changed only extraction/pdfText.ts.
- Queue: pg-boss was replaced by BullMQ + Valkey; only pipeline/queue.ts changed because startQueue()/enqueueDocument() kept their signatures.

5. The Product Database Design

Database `finerp_ap` stores the WORKFLOW - never financial truth. Separation of concerns at the database level: if the two databases disagree about money, the ERP wins.

Table	Purpose	Key design details
User	Login accounts	email unique; bcrypt passwordHash; role ADMIN/AP_CLERK/FINANCE_HEAD/CFO/EMPLOYEE
IngestedDocument	One row per incoming invoice file	source UPLOAD|EMAIL; sha256 with unique(orgId, sha256) = duplicate-file guard; storageKey to the raw PDF; erpInvoiceId/erpPoId links; status = the pipeline journey
Extraction	What the AI read	vendorName, invoiceNo, poNumber, amount, dueDate + engine (MOCK/CLAUDE/OPENROUTER) recorded for audit
ApprovalTask	A required human sign-off	requiredRole FINANCE_HEAD|CFO; status; decidedBy; decidedAt
WorkflowEvent	Append-only audit log	step, actor (SYSTEM/AI/RULE/HUMAN), message, JSON detail; never updated or deleted

The document status journey

```
RECEIVED -> QUEUED -> EXTRACTING -> MATCHED -> PENDING_APPROVAL -> APPROVED -> PAID
                        |             |            (or, if amount < Rs. 50,000:)
                        |             +--------------------> APPROVED (auto)
                        +-> NEEDS_REVIEW   a verification rule failed - human must look
                        +-> DUPLICATE      same file bytes, or same invoice number in ERP
                        +-> FAILED         crashed after all queue retries
                                           REJECTED = a human said no
```

6. Ingestion: Upload and the Gmail Adapter

One front door for every source

`ingest/ingestFile.ts` is the single entry point no matter how an invoice arrives - the dashboard upload button and the Gmail poller both call it, and a future WhatsApp/Slack/API source would too. One door means the guarantees hold for every source:

- **sha256 fingerprint first.** Identical bytes already seen for this org are rejected before any storage, AI cost, or ERP write (duplicate defence no. 1; the ERP's invoiceNo constraint is no. 2).
- **Store the original before anything else.** The untouched PDF goes to storage first (audit rule: never lose the source document), organized as org/year/month/uuid.pdf behind the S3-style adapter.
- **Then record and enqueue.** A DB row is created with status QUEUED and a job is pushed. The HTTP request returns in ~100ms; processing happens asynchronously.

The Gmail adapter - how and why

Technology: **IMAP** via the imapflow library plus mailparser to decode messages and attachments. Authentication uses a Google **App Password** (a 16-character password generated after enabling 2-Step Verification). We chose IMAP + app password over the Gmail REST API with OAuth because it needs five minutes of setup instead of a Google Cloud project, consent screens, and token-refresh plumbing - and the result is identical for a single test inbox. (Production, where each customer connects their own inbox, would use OAuth - that is an adapter swap.)

The poller (ingest/gmailPoller.ts) runs every 60 seconds:

```
connect to imap.gmail.com:993 (fresh connection per poll - no stale-socket bugs)
search UNSEEN messages
for each: download PDF attachments -> ingestFile(org, name, bytes, 'EMAIL')
mark the email as Seen ONLY after successful ingestion
any error: log it, wait for the next tick - the poller can never crash the server
```

- Marking Seen only after success means a crash mid-email leaves it unread - the next poll retries it. Same never-lose-an-invoice principle as the queue.
- Recursive setTimeout instead of setInterval guarantees a slow poll can never overlap the next one.
- Duplicate emails are harmless: the sha256 check silently skips already-ingested attachments.

7. Queuing: BullMQ + Valkey

Why a queue exists at all

Without a queue, the upload handler would run the whole pipeline synchronously. Four disasters follow: (1) a server crash mid-processing loses the invoice forever - it lived only in memory; (2) a transient failure (ERP down for 10 seconds) permanently fails an invoice because nothing retries; (3) 200 invoices arriving at 9am spawn 200 simultaneous AI calls and overload everything; (4) the uploader waits 30 seconds for a response. A queue fixes all four: jobs are persisted in the broker, retried with backoff, processed at a controlled rate, and the upload returns instantly.

The components

Role	What	Where
Broker	Valkey 8 (the Linux Foundation open-source fork of Redis, BSD-licensed) in Docker; appendonly persistence so jobs survive restarts	docker-compose.yml, port 6379
Producer	enqueueDocument(documentId) - called by ingestFile for every new document	pipeline/queue.ts
Worker	A BullMQ Worker with concurrency 1 that runs processDocument(documentId)	pipeline/queue.ts
Job payload	Just { documentId } - all real data stays in PostgreSQL; the queue carries pointers, not state	-

Why BullMQ + Valkey instead of the simpler pg-boss we started with: BullMQ is the industry-standard Node.js queue and Redis-protocol brokers are the industry default - directly relevant experience. Valkey specifically because it is fully open source (Redis changed its license in 2024) with AWS/Google backing.

Retries, idempotency, and the DLQ question

```
attempts: 3, backoff: exponential 5s -> 10s -> 20s
removeOnComplete: 100, removeOnFail: 500   (keep recent history for inspection)
```

- **Idempotency guard:** processDocument starts with 'if (doc.status !== QUEUED) return'. If a retry fires after the work already finished, it does nothing. Retries are only safe when re-running is harmless.
- **Failed set = the DLQ.** After the third failed attempt, BullMQ parks the job in its failed set and our worker handler marks the document FAILED with the reason. The queue moves on.
- **No poison-message deadlock - proven by experiment.** We uploaded a corrupted PDF followed immediately by a valid one. The corrupt job failed, was scheduled for delayed retry (which FREES the worker), the valid invoice processed to APPROVED within 10 seconds, and the corrupt one ended FAILED ('Invalid PDF structure.') after its three bounded attempts. Bounded retries + a failed-jobs parking lot = no infinite loop, no blocking.

Observability

Bull Board (mounted at localhost:5000/admin/queues) visualizes the queue live: Waiting / Active / Delayed / Completed / Failed tabs, per-job error and stack trace, and a manual Retry button. Raw broker inspection (Medis / RedisInsight against localhost:6379) works too, but shows BullMQ's internal

keys rather than job semantics.

8. Extraction: PDF-to-Text and the AI Layer

Step 1 - getting text out of the PDF

Digital PDFs carry their text inside the file. `extraction/pdfText.ts` uses **pdfjs-dist** (Mozilla's PDF engine, the one inside Firefox) to pull it out. We originally used the popular pdf-parse library; it failed on our files with 'bad XRef entry' - it is unmaintained and bundles an ancient engine. Because parsing sat behind an adapter, the swap touched one file. Scanned/photographed invoices would need OCR (e.g. AWS Textract) - that alternative path plugs into the same file.

Step 2 - the AI turns text into structured data

This is the ONLY place AI exists in the system. The interface (extraction/extractor.ts) defines the contract every engine returns:

```
{ vendorName, invoiceNo, poNumber, amount, dueDate }   // every field nullable
```

Engine	What it is	When used
MOCK	Regex over labelled lines ('Vendor:', 'Amount Due:'). Free, offline - and deliberately brittle: it demonstrates exactly why real products need an LLM	no API key set
CLAUDE	Anthropic API directly (@anthropic-ai/sdk)	ANTHROPIC_API_KEY set
OPENROUTER	One API key fronting many models (Claude/GPT/Gemini) via the OpenAI-compatible chat-completions format - called with plain fetch, no SDK. Model chosen by env var (anthropic/claude-haiku-4.5, about Rs. 0.05 per invoice)	OPENROUTER_API_KEY set

Engine selection lives in one file (extraction/index.ts): Anthropic key -> Claude; else OpenRouter key -> OpenRouter; else mock. Switching models or providers is configuration, not code.

- temperature: 0 - extraction wants determinism, not creativity.
- The prompt demands strict JSON with nulls for missing fields and 'never guess numbers'.
- **The AI's answer is treated as untrusted input.** It is parsed, type-checked field by field, stored with the engine name for audit, and then re-verified against the ERP by the rule engine before any money-adjacent action. Proof it works: an invoice written with labels the mock could never read ('Bill No.', 'Against Order', 'Grand Total (incl. GST): Rs. 1,75,000.00', a date written in words) extracted perfectly, including the Indian comma format.

9. Verification: the Rule Engine

`rules/verify.ts` is pure deterministic code - it imports no AI. It answers one question: is this invoice safe to act on? Five ordered rules, cheapest first, each with a precise human-readable failure reason:

#	Rule	Catches
1	Extraction produced vendor + invoiceNo + amount	unreadable/garbage documents
2	Vendor exists in the ERP (fuzzy name match: 'Acer India Pvt Ltd' contains 'Acer')	invoices from unknown companies - a classic fraud vector
3	Invoice references a PO, and an OPEN PO with that number exists	unsolicited bills; double-billing (a second invoice against an already-MATCHED PO finds no OPEN PO and is flagged - we caught this in testing without writing special code for it)
4	The PO belongs to the same vendor as the invoice	billing someone else's order
5	Invoice amount equals PO amount exactly	overcharging, typos, AI hallucination of numbers

Any failure sets the document to NEEDS_REVIEW with the reason, writes an audit event, and STOPS the pipeline - nothing is written to the ERP, no approval is requested, no money can move. Success hands over a verified vendor + PO pair, and the invoice is created in the ERP linked to that PO.

Why rules and not AI for this step: the checks are exact comparisons over clean data. Code is cheaper (no tokens), faster, testable, and auditable - and it cannot hallucinate. This is rule 'AI reads, rules decide' in action. The industry name for this check is the **three-way match** (PO vs goods receipt vs invoice; we implement the two-way PO-vs-invoice version, with goods receipt as a future extension).

10. Routing and Approvals

Threshold routing (rules/route.ts)

```
amount <  Rs.  50,000   -> AUTO-APPROVE          nobody is asked
Rs. 50,000 - 5,00,000   -> FINANCE_HEAD task
amount >  Rs. 5,00,000  -> CFO task
```

The auto-approve band is the product's economic core: the majority of invoices are small and clean, and they flow through with zero human seconds spent. Humans see only the big or broken ones. (v1 keeps thresholds as constants; production makes them per-organization rows plus a settings screen.)

The decision flow, and a hard-won ordering rule

When a human decides, the product updates **the ERP first**, then its own records. If the process dies between the two writes, the ERP (source of truth) is already correct and the product can re-sync; the reverse order could leave an approval recorded that the ERP never saw.

Conflict reconciliation - the out-of-band problem

Both the product AND the ERP's own UI can decide an invoice. A user rejected an invoice directly in the miniERP, then clicked Approve in the dashboard. The ERP's state machine correctly refused the illegal transition - and the product originally handled that refusal so badly the whole Node process died (unhandled promise rejection). The proper design, now implemented: catch the ERP's refusal, read the ERP's actual status, sync the product's task + document to match it, write an honest audit event ('already REJECTED directly in the ERP - records synced'), and answer 409 with that explanation. **The system of record wins; the product reconciles toward it.** Real customers do exactly this in QuickBooks, so real AP products must survive it.

11. Authentication and Role-Based Access

Mechanics

- Passwords are stored only as **bcrypt hashes** (one-way; nobody, including the operator, can read a password back). Login compares hashes.
- A successful login issues a **JWT** (12h expiry) carrying id, name, role, and organizationId. Every API call sends it as 'Authorization: Bearer ...'.
- **The tenant now comes from identity:** requireAuth verifies the token and takes organizationId from it. The earlier spoofable x-org-id header is gone - a user cannot browse another company's data by editing a header.
- Login inputs are trimmed and the email lowercased - copy-pasted credentials often carry an invisible trailing space (a real 'invalid password' bug we hit and fixed).

Roles and the permission matrix

Action	EMPLOYEE	AP_CLERK	FINANCE_HEAD	CFO	ADMIN
View documents / timelines	yes	yes	yes	yes	yes
Upload invoices	-	yes	yes	yes	yes
Approve 50k-5L (FINANCE_HEAD tasks)	-	-	yes	yes	yes
Approve > 5L (CFO tasks)	-	-	-	yes	yes
Record payment	-	-	yes	yes	yes

The core rule in code: a task may be decided by its requiredRole, or by CFO/ADMIN who outrank. The SERVER enforces everything (403s); the UI's hidden buttons and lock icons are honesty, not security. The audit log records the authenticated person - 'CFO APPROVED by Meera' is tied to a login, not typed text.

Registration

Self-service sign-up (/register) with a designation dropdown (Employee / AP Clerk / Finance Head / CFO / Admin), password minimum 8 chars, duplicate-email check, and auto-login on success. Honest caveat, documented in code: self-picking powerful roles is a demo convenience - production shrinks the self-serve list to EMPLOYEE and assigns finance roles via admin invitations (the SELF_SERVE_ROLES constant makes that a one-line change).

12. The Dashboard (Frontend)

Next.js 15 (App Router) + React 19, hand-scaffolded - no create-next-app, no Tailwind, no UI library. Plain CSS keeps the free-tier build trivially deployable and every line understandable. Design system in one globals.css: indigo accent (the ERP UI uses teal so you always know which app you are in).

- **Zero business logic by design.** The dashboard can only display and request; every rule lives server-side. A UI that cannot break the rules is the point.
- lib/api.ts is the single door to the backend: all fetches, the Bearer token, 401-redirect to /login, types. Pages never build URLs themselves.
- Pages: /login, /register, / (documents table, live via 3-second polling, drag-and-drop upload, stat cards, status filter chips), /documents/[id] (extraction card + audit timeline + Record Payment), /approvals (inbox with role-aware Approve/Reject or lock icons).
- Status badges pulse while processing; the audit timeline color-codes actors (AI purple, RULE green, HUMAN amber, SYSTEM gray) - the architecture is visible in the UI.

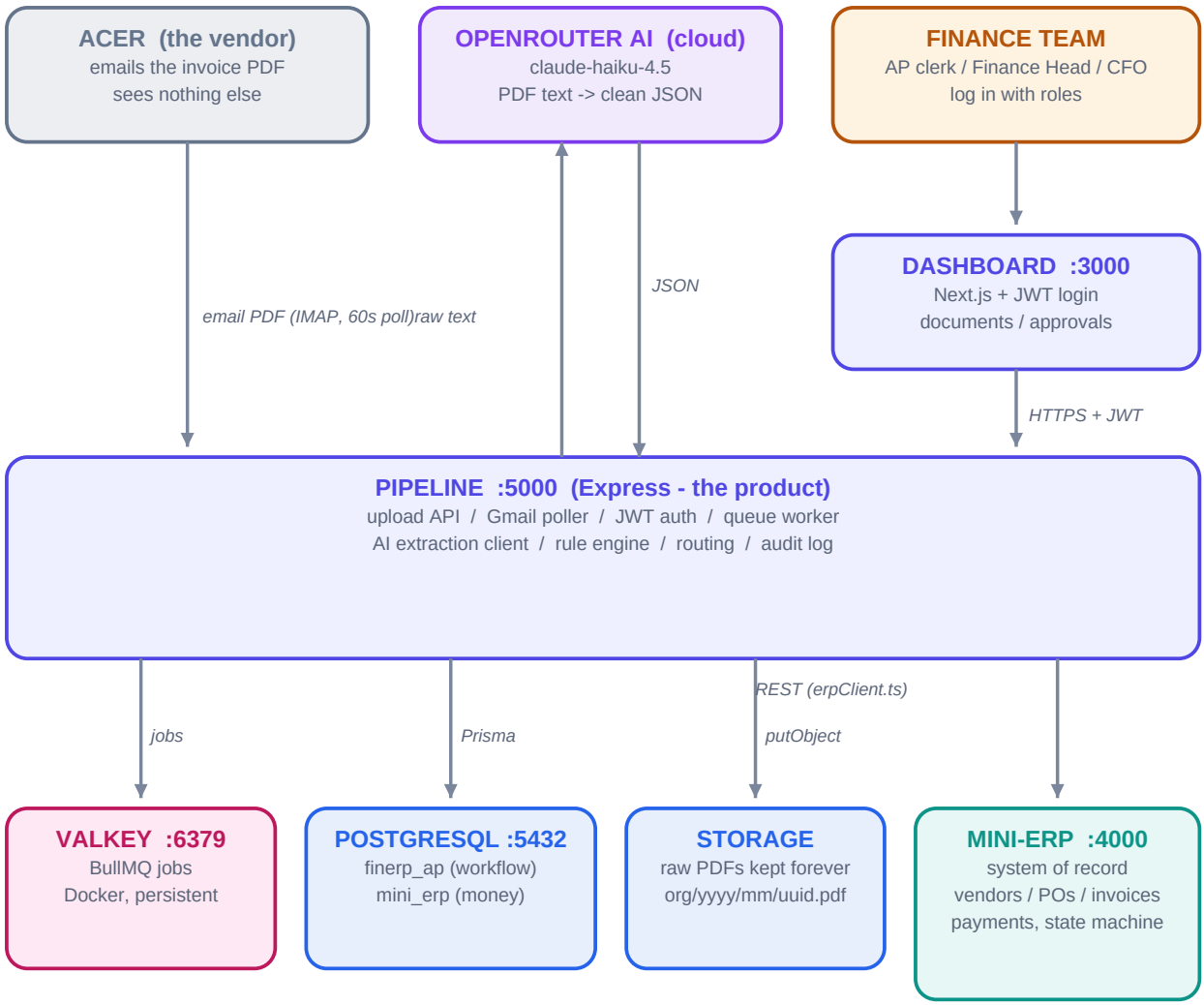
13. Audit Trail and Observability

WorkflowEvent is append-only: every pipeline step, rule verdict, human decision, and conflict reconciliation writes one row with step, actor (SYSTEM / AI / RULE / HUMAN), a human-readable message, and optional JSON detail. Rows are never updated or deleted; corrections are new events. This answers the auditor's question - who did this, when, and why - including decisions made by AI and by rules.

Tool	What it shows	Where
Dashboard timeline	Per-invoice story with actor chips	localhost:3000/documents/[id]
Bull Board	Queue states, per-job errors, retry button	localhost:5000/admin/queues
Prisma Studio	Visual table browser (the 'MongoDB Compass' of this stack)	npx prisma studio -> localhost:5555
pgAdmin 4	Raw SQL over both databases	installed with PostgreSQL
Server logs	Every audit line echoes to the pipeline terminal	npm run dev output

14. The Complete System Design

Everything the system is, on one page. Arrows show the direction data flows; colors show the kind of component (violet = AI, indigo = the product, teal = the system of record, pink = queue, blue = state, gray/amber = the outside world).



The ERP is the memory: on any conflict, the product syncs toward it.

AI (violet) touches exactly one step; everything below the pipeline is deterministic.

One invoice, end to end (the demo that ran on real email + real AI)

```
0. PO-2026-050 raised in the ERP: 2 laptops, Rs. 90,000, status OPEN
1. vendor emails ACR-5001.pdf   ->  poller ingests it (source EMAIL)
2. sha256: new  ->  stored to storage/org_demo/2026/07/<uuid>.pdf  ->  QUEUED
3. worker: pdfjs-dist -> text -> OpenRouter (claude-haiku-4.5) -> JSON  [AI]
4. rules: fields ok, vendor known, PO open, same vendor, 90,000 == 90,000  [RULES]
5. invoice created in ERP linked to PO  ->  PO auto-MATCHED
6. Rs. 90,000 is in the 50k-5L band  ->  FINANCE_HEAD ApprovalTask  [RULES]
7. Finance Head clicks Approve  ->  ERP first, then product records  [HUMAN]
8. Record Payment  ->  ERP atomic transaction: payment + invoice PAID + PO CLOSED
9. every step above wrote a WorkflowEvent  ->  visible as the audit timeline
```

The golden principles (the whole document in five lines)

- AI reads, rules decide - AI exists in exactly one step, and its output is verified before money moves.
- The ERP is the memory - single source of financial truth; on conflict, the product syncs toward it.
- One front door per concern - ingestFile for input, erpClient for the ERP, api.ts for the UI; adapters everywhere else.
- Everything retries, nothing repeats - queue with bounded backoff + idempotency guards + hash/constraint dedupe.
- Everything is audited - append-only log with the actor (SYSTEM/AI/RULE/HUMAN) on every event.

15. War Stories: Real Bugs We Hit and Fixed

Bug	Root cause	Fix / lesson
pdf-parse: 'bad XRef entry' on valid PDFs	unmaintained library with an ancient engine	swapped to pdfjs-dist; one file changed thanks to the adapter - vet a library's maintenance, not its download count
Docker Desktop crashed on startup, survived reboot	zombie Windows unix-socket files (dockerInference, engine.sock) from a crashed first run	rename the parent folder, let Docker create a fresh one - sometimes the workaround beats the 'proper' fix
OpenRouter 404 'No endpoints found'	model slug anthropic/claude-3.5-haiku had been retired	queried the live /models endpoint and picked claude-haiku-4.5; model IDs rotate - discover, don't hardcode
Documents stuck QUEUED after all retries	final-failure path never marked them FAILED	worker 'failed' handler now sets FAILED when attempts are exhausted - always design the terminal state
Whole server died when approving a rejected invoice	unhandled promise rejection kills Node 15+; routes lacked an async error wrapper	asyncHandler on every route + central error middleware - one rejected promise must never kill a process
Dashboard 'Failed to fetch' after deciding in the ERP UI	no reconciliation for out-of-band ERP changes	detect the refused transition, sync to ERP truth, answer 409 with an explanation - system of record wins
'Invalid password' with correct credentials	copy-paste carried a trailing space; exact string compare	trim inputs, lowercase emails - a top real-world login support ticket
Next.js 500s / EADDRINUSE during dev	stale .next build cache; orphaned child process holding the port	rm -rf .next; find and kill the port holder - know your dev-loop failure modes

16. What Is Not Built Yet (the Roadmap)

Feature	Where it slots in	Size
Needs-review resolution (edit extraction, requeue from the dashboard)	new endpoint in app.ts + a form on the detail page	small
Free-tier deployment (Vercel + Railway/Render + Neon + Upstash)	everything is already env-driven; storage must move to R2/S3 first (lib/storage.ts)	medium
Invitation-based role assignment	shrink SELF_SERVE_ROLES to EMPLOYEE; add admin invite flow	small
OCR for scanned invoices	alternative path inside extraction/pdfText.ts (e.g. Textract)	medium
Per-organization approval thresholds	rules/route.ts reads a DB table + settings UI	small
Real ERP connectors (QuickBooks/Xero)	new client file mirroring erpClient.ts; OAuth per customer	large
Accounts Receivable side (reminders, collections, cash application)	new pipeline steps reusing every pattern in this document	large
Gmail push (watch + Pub/Sub) instead of polling	swap inside gmailPoller.ts	medium

Built step by step, tested end to end at every stage: a working replica of a modern AP automation product - miniERP as the system of record, finErpAP as the pipeline and dashboard, with real email ingestion, real AI extraction, deterministic verification, role-based approvals, and a complete audit trail. Every choice in this document was made for a reason you can now explain.